

Vulnerability 3: DOM-Based XSS in Search

CWE: CWE-79 - Improper Neutralization of Input During Web Page Generation

OWASP: A03:2021 - Injection

CVSS: 6.1 (Medium)

Location: roufrontend/src/app/search-result/search-result.component.ts

Component: Frontend - Search result page (Angular SPA)

Description

The product search page reads the q query parameter from the URL and injects it directly into the DOM using Angular's `bypassSecurityTrustHtml()` method. This explicitly disables Angular's built-in HTML sanitization, meaning any HTML or JavaScript in the search term is rendered as executable content. An attacker can craft a URL containing a malicious payload that executes in the victim's browser when the link is opened.

Unlike server-reflected XSS, this vulnerability lives entirely in client-side code. The server never sees the payload in a dangerous form the DOM manipulation happens in the browser after the page loads.

Vulnerable code (Backend):

```
this.searchValue = queryParams
```

```
if (this.gridDataSourceSubscription) {  
  this.gridDataSourceSubscription.unsubscribe()  
}
```

```
1  filterTable () {  
2    let queryParams: string = this.route.snapshot.queryParams.q  
3    if (queryParams) {  
4      queryParams = queryParams.trim()  
5      this.ngZone.runOutsideAngular(() => { // vuln-code-snippet hide-start  
6        this.io.socket().emit('verifyLocalXssChallenge', queryParams)  
7      }) // vuln-code-snippet hide-end  
8      this.dataSource.filter = queryParams.toLowerCase()  
9      this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParams)  
10     if (this.gridDataSourceSubscription) {  
11       this.gridDataSourceSubscription.unsubscribe()  
12     }  
13     this.gridDataSourceSubscription = this.gridDataSource.subscribe((result: ProductTableEntry[]) => {  
14       if (result.length === 0) {  
15         this.emptyState = true  
16       } else {  
17         this.emptyState = false  
18       }  
19     })  
20   } else {  
21     this.dataSource.filter = ''  
22     this.searchValue = undefined  
23     this.emptyState = false  
24   }  
25 }
```

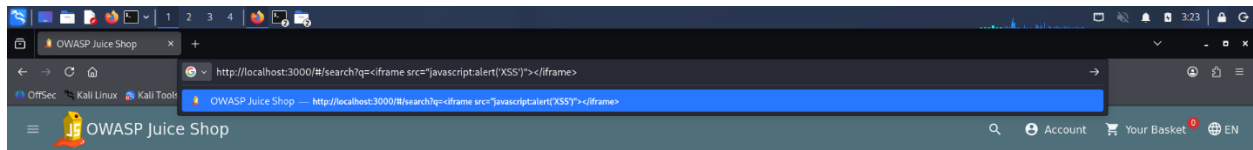
```
1  this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParams)
```

The `bypassSecurityTrustHtml()` call tells Angular to treat the string as safe HTML. Combined with a template binding like `[innerHTML]="searchValue"`, this renders attacker-controlled HTML directly into the DOM. The `queryParams.q` value comes from `this.route.snapshot.queryParams.q` the `q` parameter in the URL, fully attacker controlled.

Exploit payload: `<iframe src="javascript:alert('XSS')"></iframe>`

`http://localhost:3000/#/search?q=<iframe src="javascript:alert('XSS')"></iframe>`

1. Craft the URL with the payload in the `q` parameter
2. Send it to a victim or open it directly in the browser
3. The search page loads and the payload is injected into the DOM
4. The JavaScript in the `src` attribute executes, triggering the alert

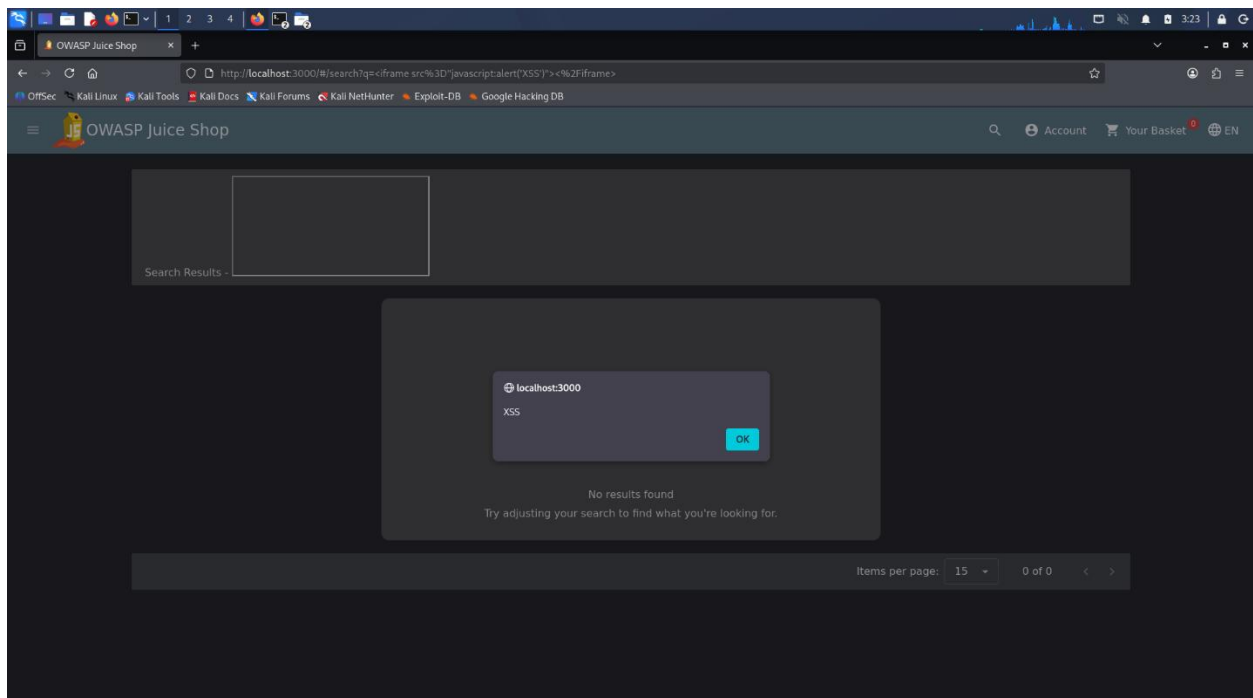


The Vulnerability:

Component	Role
URL <code>q</code> parameter	Attacker-controlled input source
<code>bypassSecurityTrustHtml()</code>	Disables Angular's XSS protection
<code>[innerHTML]</code> binding in template	Renders the string as executable HTML
<code><iframe src="javascript:..."></code>	Executes JavaScript when rendered

Evidence - Before Fix

- The search page loads, and the payload is injected into the DOM via Angular's `bypassSecurityTrustHtml()` method, which disables built-in HTML sanitization.
- The JavaScript in the `src` attribute executes immediately in the browser, triggering the alert dialog with the message **"XSS"** over the Juice Shop search page.
- Below showing the alert dialog displayed on top of the search results page, confirming arbitrary JavaScript execution in the victim's browser session.



SAST Detection - Before Fix

Tool: Semgrep

Rule: Custom rule targeting `bypassSecurityTrustHtml` on user input

Run against the unmodified `roufrontend/src/app/search-result/search-result.component.ts`, Semgrep flags the interpolated query. Capture the finding count and the specific rule ID and line number flagged.

```
(semgrep-venv)-(kali@kali)-[~/juice-shop-lab/juice-shop]
$ cd ~/juice-shop-lab/juice-shop
semgrep scan --config .semgrep-rules/ --metrics=off \
  > frontend/src/app/search-result/search-result.component.ts \
  > --json --output=semgrep-xss-before.json

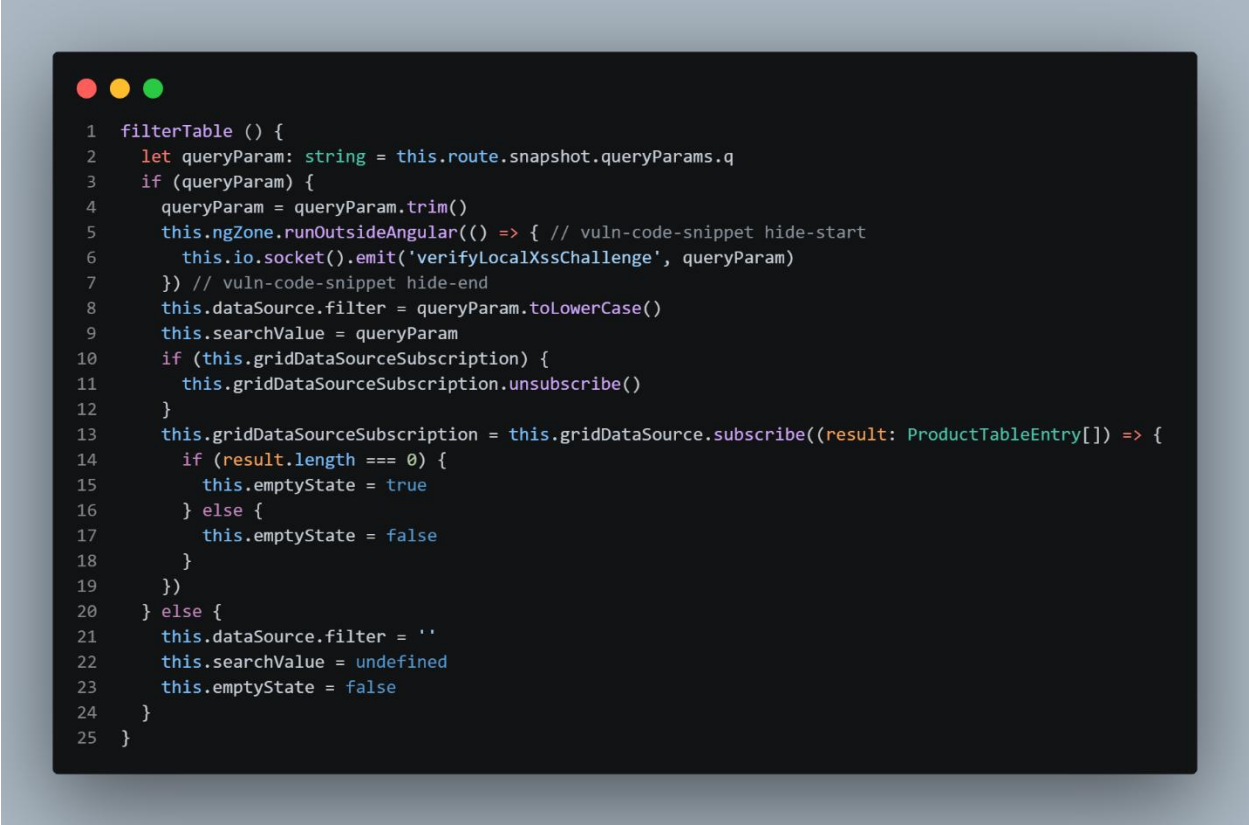
ooo Node.js version v24.19.0 (success)
Semgrep CLI OS linux (success)
CPU x64 (success)
info: Configuration default validated (success)
? Loading rules ... 21 of 21 are initialized (success)
Scanning 1 file (only git-tracked) with 2 Code rules: d (success)
info: Required file server.js is present (success)
in CODE RULES d file index.html is present (success)
in Scanning 1 file with 2 ts rules. present (success)
info: Required file main.js is present (success)
in SUPPLY CHAIN RULES polyfills.js is present (success)
info: Required file matching /^hacking-instructor-.+\.js$/ is
in No rules to run. available (success)
info: Domain https://www.alchemy.com/ is reachable (success)
warn: Environment variable ALCHEMY_API_KEY is not present (WARN)
wa PROGRESS the Honey Pot challenge will not work as intended
warn: "Wallet Depletion" challenge will not work as intended w
in https://howto-llm.owasp-juice.shop/ 100% 0:00:00
warn: Domain http://localhost:11434/v1 is not reachable (WARN)
warn: "Chatbot Prompt Injection" challenge will not work as in
warn: "Greedy Chatbot Manipulation" challenge will not work as
ing challenge will not work as intended witho
Prompt Extraction challenge will not work as in
s://howto-llm.owasp-juice.shop for instruction
[✓] Scan completed successfully. 3000
• Findings: 2 (2 blocking)
• Rules run: 2
• Targets scanned: 1
• Parsed lines: ~100.0%
• No ignore information available
Ran 2 rules on 1 file: 2 findings.
```

Secure Coding Fix

Remove the bypass call and rely on Angular's built-in output encoding.

```
this.searchValue = queryParams
```

```
if (this.gridDataSourceSubscription) {  
  this.gridDataSourceSubscription.unsubscribe()  
}
```



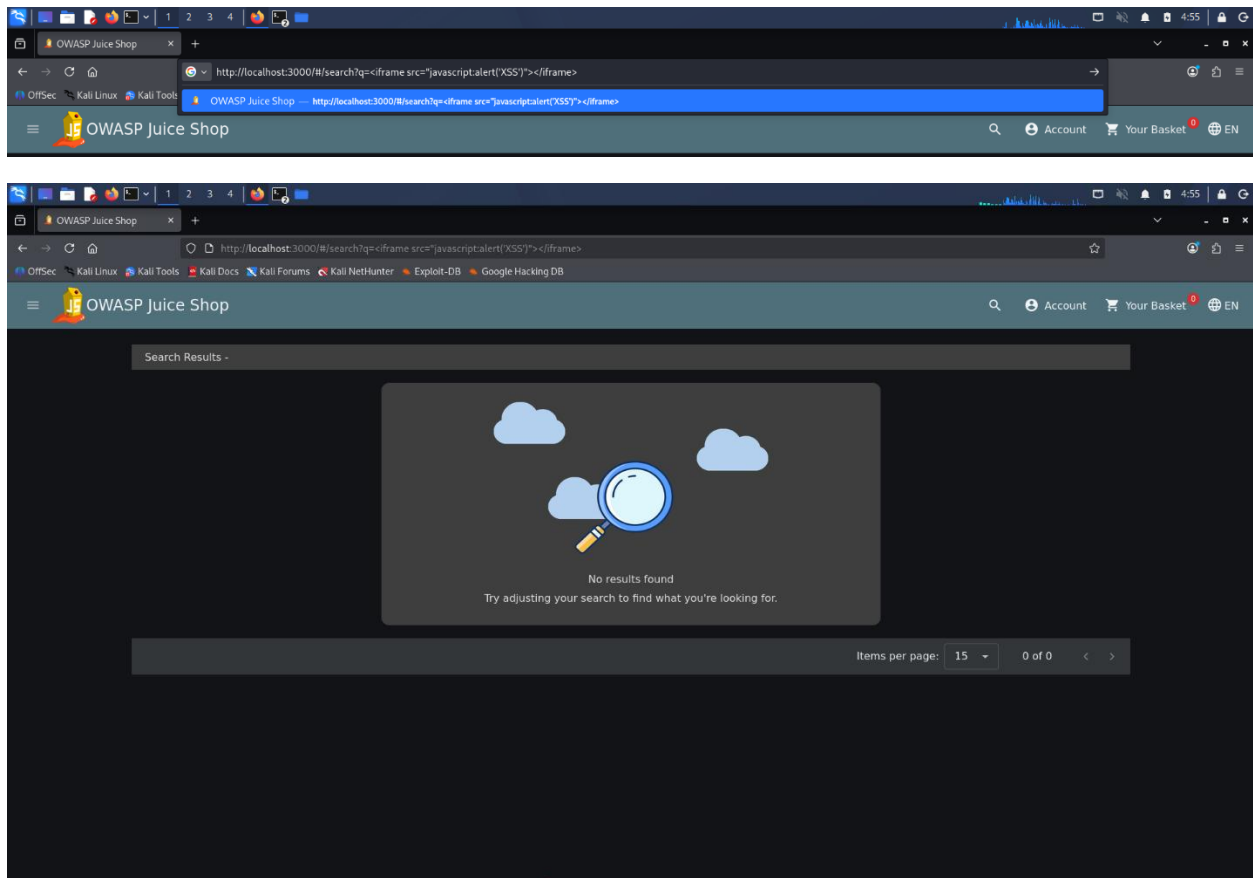
```
1  filterTable () {  
2    let queryParams: string = this.route.snapshot.queryParams.q  
3    if (queryParams) {  
4      queryParams = queryParams.trim()  
5      this.ngZone.runOutsideAngular(() => { // vuln-code-snippet hide-start  
6        this.io.socket().emit('verifyLocalXssChallenge', queryParams)  
7      }) // vuln-code-snippet hide-end  
8      this.dataSource.filter = queryParams.toLowerCase()  
9      this.searchValue = queryParams  
10     if (this.gridDataSourceSubscription) {  
11       this.gridDataSourceSubscription.unsubscribe()  
12     }  
13     this.gridDataSourceSubscription = this.gridDataSource.subscribe((result: ProductTableEntry[]) => {  
14       if (result.length === 0) {  
15         this.emptyState = true  
16       } else {  
17         this.emptyState = false  
18       }  
19     })  
20   } else {  
21     this.dataSource.filter = ''  
22     this.searchValue = undefined  
23     this.emptyState = false  
24   }  
25 }
```



```
1  this.searchValue = queryParams
2      if (this.gridDataSourceSubscription) {
3          this.gridDataSourceSubscription.unsubscribe()
4      }
```

Why this works: Angular's `{{ }}` interpolation performs context-aware output encoding. Characters like `<`, `>`, `&`, and `"` are escaped to safe HTML entities before insertion into the DOM. The payload is displayed as text rather than parsed as HTML, so no script executes.

Evidence - After Fix



SAST Detection - After Fix

Re-running the same scan on the patched roufrontend/src/app/search-result/search-result.component.ts shows the SQL injection finding is gone.

```
(semgrep-venv)-(kali@kali)-[~/juice-shop-lab/juice-shop]
$ cd ~/juice-shop-lab/juice-shop
semgrep scan --config .semgrep-rules/ --metrics=off \
  frontend/src/app/search-result/search-result.component.ts \
  --json --output=semgrep-xss-after.json
```

Semgrep CLI

```
 Loading rules ...
Scanning 1 file (only git-tracked) with 2 Code rules:

CODE RULES
Scanning 1 file with 2 ts rules.

SUPPLY CHAIN RULES
No rules to run.

PROGRESS
100% 0:00:00
```

Scan Summary

```
✓ Scan completed successfully.
• Findings: 0 (0 blocking)
• Rules run: 2
• Targets scanned: 1
• Parsed lines: ~100.0%
• No ignore information available
Ran 2 rules on 1 file: 0 findings.

✦ If Semgrep missed a finding, please send us feedback to let us know!
See https://semgrep.dev/docs/reporting-false-negatives/
```

Before / After Summary

Metric	Before Fix	After Fix
Semgrep finding count on component	1	0
Exploit payload renders	Alert fires	Rendered as text
DOM sink	bypassSecurityTrustHtml() + [innerHTML]	Interpolation {{ }}
Root cause present	Yes	No
CWE-79 compliant	No	Yes